GCAM emotions, GPU graph visualization, graph DB and temporal GNN: revised plan

Prepared: Saturday, Oct 17, 2026, 18:20–18:35 (Bogotá, UTC-5) for the 30X "Radar de contenido" hackathon (deadline Sun Oct 18, 12:00 UTC-5).

Trigger: the user has about **US$250 of Google Cloud credit** and wants:

- GCAM over a restricted (or larger) window;
- a serious look at graph visualization, with GPU acceleration explicitly favored;
- a graph DB for the demo;
- a deeper temporal GNN (TGNN) plan.

Labels used throughout:

- **(measured)** = we ran it today.
- **(verified)** = primary source read today; the source is named.
- **(estimate)** = our projection.
- **UNVERIFIED** = not confirmed.

Companion files: `MASTER_PLAN.md` (updated), `bq_sql/s7_gcam_monthly.sql`, `s7d_gcam_daily.sql`, `s9_gcam_topic.sql`, `s10_graph_viz_export.sql`.

---

0. TL;DR

1. **GCAM is cheap enough to run on all 2 years.**
   - BigQuery bills the GCAM column in full for every touched partition, so cost depends on **days, not rows**.
   - At the measured 9,172 B/row and 245k–331k rows/day:

Window	Data scanned	Cost at $6.25/TiB beyond the free tier
30 days	67–91 GB	**$0.38–0.52**
90 days	0.18–0.25 TiB	**$1.2–1.6**
2 years	**1.5–2.2 TiB**	**$9–14**

   - **Recommendation:**
     - Run the **last ~108 days first** (Oct, Sep, Aug, Jul 2026) for the demo, about $2.
     - Then the **full 2 years overnight**, newest month first, about $9–15 total. That is under 6% of the credit.
   - Each monthly job needs **Maximum bytes billed raised to 150 GiB** for that job only.
   - The heavy column is read **once per month** into a narrow table: `radar.gcam_ai`, 46 emotion dimensions plus word counts, all verified in the GCAM master codebook.
   - The same scan also builds `radar.gcam_daily_baseline` (the emotional climate of all news) **at no extra GCAM cost**.
2. **Visualization: two tiers, GPU-first.**
   - **Tier 1, "wow" view: Graphistry Hub (primary).** Layout runs on Graphistry's server GPUs; rendering runs on WebGL2 in the browser. It shows the whole 90-day AI co-mention graph with a timebar, embedded in our app by iframe URL.
     - Size: about 0.1–0.33M nodes and 0.3–0.6M edges with n ≥ 3 (estimate).
     - Graphistry 2.53.0 (Sep 3, 2026, verified on its blog) opens 10M edges in 537 MB of browser memory, though it is "still choppy" at that size.
     - **Backup: cosmos.gl** (MIT, OpenJS; v3 shipped Jun 25, 2026, verified), fed with **coordinates precomputed by RAPIDS cuGraph ForceAtlas2 on a Modal GPU**. This keeps the demo laptop's GPU on rendering only.
   - **Tier 2, "story" view: sigma.js v3 + graphology** (MIT; repo pushed Oct 16, 2026, verified), using the pruned top-K weekly graph from S10.
     - Size: ≤ 400 nodes and ≤ 2,500 edges per week.
     - Interactions: a week slider, nodes colored by tone/arousal, NPMI edge tooltips, Leiden community colors, and click-to-see evidence articles.
3. **Graph DB: none for computation; optional for the demo.** Memgraph Lab Community (free) in Docker on the box gives a 2-minute "ask the graph in Cypher" moment.
   - Its renderer defaults are 3,500 nodes and 8,500 edges (verified, Memgraph docs).
   - AuraDB Free (200k nodes / 400k relationships per the current Neo4j docs) is the fallback.
4. **TGNN: a real but gated stretch.**
   - **Library: TGM** (`tgm-team/tgm`, MIT, beta, arXiv:2510.07586, pushed Sep 22, 2026). It implements EdgeBank, GraphMixer, TGAT, TGN, DyGFormer, TPNet, PopTrack and more.
   - **Evaluation:** TGB 2.3.0 (Jul 27, 2026) for the evaluator and negative sampling.
   - **Tasks:**
     - (a) future entity–topic link prediction, scored with MRR on **new** links;
     - (b) next-week node traction, scored with NDCG@10.
   - **Compute:** about $2–8 per model on a Modal A10 or L4 (estimate).

- **Go/no-go at 07:00:** the TGNN only goes into the demo if it beats EdgeBank, persistence and PopTrack on new-link MRR by ≥ 10% relative across 3 seeds **and** we can show its top contributing past interactions. Otherwise we show an honest baseline table.

5. **"Graphiphy":** no graph-visualization product by that name exists.
    - **Graphify** is real, but it is a codebase-to-knowledge-graph skill for AI coding assistants (see §2.1), not a data-viz tool for our graph.
    - The likely intended tools are **Graphistry** (named by the user) and/or **Gephi / Gephi Lite**.
    - Gephi Lite has **no timeline**: it offers only a date-range filter (verified, Gephi Lite docs). Desktop Gephi has a timeline but can't be embedded.

---

1. GCAM with the $250 credit

1.1 Cost math (recomputed)

Billing rule (verified earlier today, cloud.google.com):

- On-demand queries cost **$6.25/TiB** after the first **1 TiB/month free**.
- Bytes = the full size of every referenced column across the pruned partitions.
- `WHERE` / `IN` filters don't reduce bytes. `gkg_partitioned` is partitioned by ingestion day, so cost scales with **days scanned**.

Bytes per row of the S7 parse statement:

Column	B/row
GCAM	9,172
GKGRECORDID	21
DATE	8
TranslationInfo	18
Total	≈ **9,219** (measured column sizes, Oct 16 sample)

Rows per day: 331k from the raw files (measured), or about 245k if BigQuery holds the 74% the user observed. Older months have about 10–20% more rows (estimate from file sizes).

Window	Rows (245k–331k/day)	Bytes scanned	TiB	$ if fully beyond the free tier
30 days	7.4–9.9M	68–92 GB	0.062–0.083	$0.39–0.52
60 days	14.7–19.9M	136–183 GB	0.12–0.17	$0.77–1.04
90 days	22–30M	203–275 GB	0.18–0.25	$1.15–1.56
~108 days (Jul 1 → Oct 17)	26–36M	244–330 GB	0.22–0.30	**$1.4–1.9**
2 years (730 days, +10% for older months)	197–266M	1.65–2.45 TB	**1.5–2.2**	**$9.4–14**

The parent's numbers check out. ~93 GB per 30 days is right, and 2 years comes to about 2 TiB, or about $12. The month's free 1 TiB is mostly consumed by the graph-ready backfill (0.42–0.61 TiB), so assume GCAM is billed in full.

Extra reads of our own tables:

- Statements 2–3 of each monthly run read the temp table, about 3–4 GB each.
- S9 and S10 read only our own tables.
- Extra GCAM-related reads total ~0.2 TiB over 2 years, about $1.3 (estimate).

Daily maintenance (S7d): 2 ingestion partitions ≈ 6 GB/day ≈ 0.18 TiB/month ≈ **$1.1/month** (estimate).

1.2 Recommendation

1. Dry-run `s7_gcam_monthly.sql` for **October 2026** (partial month). Confirm the estimate is ≤ ~60 GB.
2. In Query settings, set Maximum bytes billed to **161061273600 (150 GiB)** for this job only.
3. Run October, then September, August and July. That is about 108 days for ~$2, enough for every demo view: 90-day graph, emotion series, framing angles.
4. Overnight: run the remaining 20 months newest-first.
    - The project's **0.7 TiB/day query quota** allows about 7 monthly runs per day. Either raise it temporarily to 2.5 TiB or spread the runs over ~3 nights.
    - Keep the $1 budget alert, and **add alerts at $25 and $50**.
5. Restore the 50 GiB cap afterwards. Schedule S7d daily after S6 with a 20 GiB cap.

UNVERIFIED: whether the $250 credit covers BigQuery on-demand analysis. Most Google Cloud promotional and trial credits do, but some are product-scoped. Check *Billing → Credits → scope* before the 2-year run.

1.3 Which GCAM dimensions to keep (all verified in `src/gcam.txt`, the GCAM master codebook)

Coverage was measured on **6,184 GKG rows** from 9 files dated Oct 17, 22:00–23:30 UTC (17:00–18:30 UTC-5). Output is in `gdelt_poc/gcam_coverage_out.txt`. The AI subset in that sample is small (65 rows), so AI-specific coverage is indicative only.

Group	Keys (dictionary)	Why	Coverage (all rows)
Word counts	`wc`, `nwc` (native word count when native dictionaries fire)	Denominators	100% / 52%
Valence / arousal / dominance	v19.1–v19.3 (ANEW 2010, scored 1–9)	**Arousal is the best single proxy for the "both extremes" thesis**	~100%
Overall sentiment	v21.1 Hedonometer, v26.1 VADER, v10.1/v10.2 SentiWordNet, c3.1/c3.2 Lexicoder neg/pos	Cross-check of V2Tone	98–100%
Discrete emotions (LIWC)	c5.31 Sad, c5.32 Anger, c5.33 Anxiety, c5.34 NegEmo, c5.35 PosEmo	Fear/anxiety framing	48–95%
Emotions (WordNet-Affect 1.1)	c15.7 alarm, c15.20 anticipation, c15.22 anxiety, c15.30 awe, c15.57 confidence, c15.103 enthusiasm, c15.147 hopefulness, c15.160 insecurity, c15.167 joy, c15.198 negative-fear, c15.212 positive-hope, c15.233 security, c15.254 surprise	The fear ↔ hope/awe axis; anticipation; confidence/security as a **trust proxy**	3–34%
Roget's (1911)	c9.896 HOPE, c9.898 FEAR	Dense fear/hope counts	50–57%
General Inquirer	c2.17 Arousal, c2.47 EMOT	Second arousal measure	~80%
Business tone (Loughran–McDonald)	c6.6 Uncertainty, c6.1 Litigious	Fits an executive audience ("uncertainty around AI regulation")	71–77%
Moral Foundations	c25.2 HarmVice, c25.4 FairnessVice, c25.11 MoralityGeneral	Moral-outrage proxy	7–42%
Spanish (native)	v22.1/v22.2 Spanish ANEW valence/arousal; v23.17/v23.18 ML-SENTICON Spanish level-1 pos/neg; v24.9 Hedonometer Spanish	**Native-language** emotion for Spanish sources	**99%** of Spanish-source rows (v22.*)
Portuguese (native)	c27.1 Emotion, c27.2 ProSocial (Detry/Lopo/Hogenraad); v24.7 Hedonometer Portuguese	Native Portuguese emotion	**98%** of Portuguese-source rows

No "trust" or NRC-style 8-emotion dictionary (NRC EmoLex) is in GCAM; we checked the codebook. Trust is therefore proxied by confidence + security (WordNet-Affect), and anticipation comes from c15.20. Treat both as proxies.

Native vs. translated text (verified, GKG 2.1 codebook, V2GCAM section):

- Non-English dictionaries run on the native text. When they fire, an `nwc` entry gives the native word count.
- English dictionaries run on the machine-translated English.
- This resolves an item marked unverified earlier: the Spanish ANEW codes do fire on Spanish originals, measured at 99%.

1.4 Parsing GCAM in SQL into a narrow table (read once)

GCAM is `wc:N,c1.1:x,...,v19.1:y,...`. Only dimensions that matched appear, so an absent `c` key means 0 and an absent `v` key means NULL.

`s7_gcam_monthly.sql` does three things in one script:

1. **The only expensive statement.** It parses GCAM for **all rows** of the month into a temp table, using a per-row correlated struct subquery. There is no global shuffle; the same pattern is applied for every key.

```
(SELECT AS STRUCT
   MAX(IF(k = 'wc', v, NULL)) AS wc,
   IFNULL(MAX(IF(k = 'c5.33', v, NULL)), 0) AS liwc_anxiety,
   MAX(IF(k = 'v19.2', v, NULL)) AS anew_arousal
 FROM (SELECT SPLIT(p, ':')[SAFE_OFFSET(0)] AS k, SAFE_CAST(SPLIT(p, ':')[SAFE_OFFSET(1)] AS FLOAT64) AS v
       FROM UNNEST(SPLIT(GCAM, ',')) AS p)
 WHERE k IN ('wc', 'c5.33', 'v19.2')) AS g
```

2. Insert the rows that exist in `radar.gkg_ai` (AI candidates plus the 0.2% control sample) into **`radar.gcam_ai`**, partitioned by dt and clustered by lang_bucket. Size is ~3.9M rows × ~400 B ≈ 1.6 GB for 2 years (estimate).
3. Aggregate **all** rows into **`radar.gcam_daily_baseline`** (dt × lang_bucket). This free ride gives "AI coverage vs. all news that day", so news-wide mood swings drop out.

Downstream:

- **`s9_gcam_topic.sql`** builds `radar.gcam_daily_topic` (dt × topic × lang × latam) with:
  - densities per 1,000 words: anxiety, fear, hope, awe/surprise, anticipation, trust proxy, uncertainty, anger, moral outrage;
  - mean arousal/valence and a high-arousal share;
  - relative-to-baseline columns.
- It also builds **`radar.gcam_topic_rolling`**: 7-day means, z-scores vs. days 34..7 back, and `z_fear_and_hope_7d`, the "both extremes rising together" signal.
- `s7d_gcam_daily.sql` is the daily incremental version.

All files pass a sqlglot BigQuery syntax check (measured). They have not yet been dry-run on BigQuery.

1.5 Licensing

- **GCAM scores** are GDELT open data; cite GDELT.
- **Each dictionary** has its own academic citation (the `DictionaryCitation` column of the codebook) and its own licence. LIWC, for example, is a proprietary dictionary.
- **We only store and display GDELT's published per-article scores**, never the dictionaries. On the about page we cite GDELT plus the dictionaries we use: ANEW, LIWC, WordNet-Affect, Roget, Loughran–McDonald, MFD, Spanish ANEW, ML-SENTICON, Hedonometer,

VADER.
- Terms for redistributing derived scores are UNVERIFIED. We show aggregates and per-article values only in the demo.

---

2. Graph visualization: GPU-first, two tiers

2.1 What "graphiphy" probably means

- **No graph-visualization product named "Graphiphy" exists.** We found:
  - a function `graphiphy()` in the RivGraph Python library (it converts river networks to NetworkX);
  - a wiki page titled "Graphiphy" in ESO's ELT software docs;
  - blog posts that misspell **Graphify**'s `/graphify` command as "/graphiphy".
- **Graphify** (`Graphify-Labs/graphify`, Apache-2.0, created Apr 2026; the repo listing showed ~128k stars, read via search today) turns a **codebase and its docs** into a queryable knowledge graph for Claude Code, Codex, Gemini CLI and similar tools. It emits `graph.html` and `graph.json`.
  - It is useful to the team's coding agents for navigating the repo.
  - It is **not** a tool for visualizing our GDELT graph.
- **Likely intended tools:** Graphistry (the user's other suggestion) or Gephi / Gephi Lite. All are compared below.

2.2 Why GPU, and where the GPU runs

- Force-directed layout costs roughly O(V log V) per iteration with Barnes-Hut. Hundreds of thousands of nodes need a GPU for interactive layout.
- Rendering more than ~100k edges smoothly needs WebGL/WebGL2 instancing.
- There are three places a GPU can run:

Where	Who	What it does	Demo-laptop risk
Graphistry's cloud GPUs (Hub, shared)	Graphistry	Layout, clustering, filtering, streaming	Low for compute; needs a good network. Hub "resource limits" apply (verified, get-started page)
The client browser GPU (WebGL2)	cosmos.gl, sigma.js, deck.gl, Graphistry's renderer	Rendering (plus layout for cosmos.gl)	**Medium-high:** integrated laptop GPUs, hardware acceleration disabled, projector resolution
Our GPU (Modal A10 $1.10/h, L4 $0.80/h, verified; Hyperbolic H100 $3.19/h via tracker)	RAPIDS cuGraph	Precompute ForceAtlas2 coordinates, Leiden, PageRank per weekly snapshot	None at demo time: coordinates ship as static files

cuGraph facts (verified, NVIDIA cuGraph 26.10 docs and the RAPIDS blog):

- `cugraph.force_atlas2(G, max_iter=..., pos_list=...)` returns x/y positions.
- It is single-GPU, with peak memory of 30×V.
- 50–100 iterations give good short-term quality.
- `pos_list` warm-starts from the previous week, so animation frames stay stable.
- RAPIDS reported **4.8 s** for a graph of 706,529 vertices and 1.24M edges on a V100.
- Leiden, Louvain, PageRank, betweenness and k-core are multi-GPU.

2.3 Size of the graph we would draw

Measured on the PoC AI rows (`gdelt_poc/graph_size.py`, output in `graph_size_out.txt`):

- 6,544 AI documents with entities.
- 10,487 distinct entities: 5,830 orgs, 4,470 persons, 172 locations, 15 themes.
- 6.5 entities per document on average (P90 13).
- 107,862 distinct co-mention pairs.
- **10,620 pairs with n ≥ 3** (9,112 with NPMI > 0.1).
- Backbone of 1,318 nodes.
- The "AI agents" slice alone: 354 documents, 772 entities, 474 pairs with n ≥ 3.

Extrapolation (estimate; Heaps fit β = 0.82 for entities, 0.96 for pairs, fitted on subsamples; small-sample Heaps fits overstate growth, so treat these as upper bounds):

Window	AI docs (4.7k/day)	Distinct entities	Pairs n ≥ 3	Co-mention events (≈16.5 pairs/doc)
60 days	~0.28M	≤ 0.23M	≤ 0.39M	~4.6M
90 days	~0.42M	**≤ 0.33M**	**≤ 0.58M**	**~7M**
2 years	~3.4M	≤ 1.8M	≤ 4.3M	~56M

Tier 1 (GPU wow):

- The 90-day graph with n ≥ 3 is about 0.1–0.33M nodes and 0.3–0.6M edges, comfortably inside Graphistry's interactive range (vendor: "most older client GPUs smoothly support 100K–1MM elements"; 2.53.0 opens 10M edges).
- It is also inside cosmos.gl's ">1 million nodes and links" claim (OpenJS).
- 2 years at n ≥ 3 (≤ 4.3M edges) is possible on Graphistry but choppy. **Use 90 days for the live demo.**

Tier 2 (story) pruning budget: implemented in `s10_graph_viz_export.sql`.

1. Per week, keep edges with n ≥ 5 and NPMI ≥ 0.2.
2. Keep the top 400 nodes by weighted degree.
3. Keep only edges between kept nodes, up to 2,500 per week, ranked by NPMI × ln(1+n).
4. For the topic ego view, take the 1-hop neighborhood of the topic node, capped at 80 nodes and 300 edges.

That stays readable on a projector. (These thresholds are estimates; tune them by eye.)

2.4 Tool comparison

Scale figures are vendor-quoted unless marked measured. Licences come from GitHub metadata today unless noted.

Tool	GPU where	Interactive scale (quoted)	Temporal animation	Embed in Lovable/Netlify	License / cost	Setup	Explainability affordances
Graphistry Hub + PyGraphistry	**Server GPU** layout (RAPIDS) + browser WebGL2 render	10M edges load in 537 MB browser memory (2.53.0, Sep 3, 2026, verified); "100K–1MM elements smooth on older GPUs" (graphistry-js README)	**Timebar** for datetime columns (Graphistry blog on time-animated graphs; exact controls UNVERIFIED) plus a histogram filter	**iframe URL** (`hub.graphistry.com/graph/graph.html?dataset=…&play=0&splashAfter=…`, verified REST docs); React component `graphistry-js`	PyGraphistry **BSD-3**; Hub **free tier**, shared GPU, "resource limits", positioned for open data (verified). Private GPU server from $1.47/h on AWS (verified)	30–60 min	Point inspector with all node/edge attributes (put evidence URLs, tone, NPMI in attributes); color/size by column; filters; legends. **Weak** at custom side panels
cosmos.gl (engine) / **Cosmograph** (app)	**Browser GPU** (layout + render; luma.gl/WebGL2 in v3)	">1 million nodes and links" (OpenJS, May 2025)	v3 animates position/color transitions (verified, Jun 25, 2026); Cosmograph app has a timeline (UNVERIFIED details)	npm package; React wrapper (UNVERIFIED for v3)	cosmos.gl **MIT** (`cosmosgl/graph`, pushed Oct 16, 2026). Cosmograph app **CC BY-NC 4.0**; commercial use needs Pro/Team (verified)	1–3 h	Custom: we write the tooltip and side panel. Fixed coordinates possible (`enableSimulation` off; parameter name UNVERIFIED for v3)
RAPIDS cuGraph (compute only)	**Our GPU** (Modal/Hyperbolic)	~200M vertices for FA2 on a 32 GB V100 (RAPIDS blog estimate); 4.8 s for 0.7M/1.2M	Warm-started weekly layouts (`pos_list`) give stable frames	Outputs Parquet/JSON for cosmos.gl, sigma.js or Graphistry	Apache-2.0 (not re-checked today); GPU cost only	1–2 h (image build)	Metrics (PageRank, Leiden, betweenness) become explanation fields
sigma.js v3 + graphology (+ FA2 worker)	Browser WebGL	Comfortable to ~10k–50k nodes/edges (community guidance; UNVERIFIED for our data)	No built-in timeline; we animate between weekly snapshots (tween positions/colors)	npm/React (`@react-sigma`), static JSON on Netlify	**MIT** (pushed Oct 16, 2026)	2–3 h	**Best custom affordances:** hover/click handlers, side panel with evidence, edge labels (NPMI), community colors, search
Gephi (desktop 0.10)	CPU (OpenGL render)	~100k nodes desktop (folklore)	**Timeline** for dynamic GEXF	Not embeddable; export to Retina or video	GPL/CDDL, free	1 h	Great for an analyst, poor for a live web demo
Gephi Lite v1.1 (Oct 6, 2026)	Browser (sigma.js)	Small/medium graphs	**No timeline**; only a date-range filter (verified docs); dynamic GEXF unsupported	Web app; workspace JSON; no first-class embed	GPL-3.0, free	15 min	Dynamic captions, filters, geographic layout (new in v1.1)
Retina (OuestWare)	Browser (sigma.js)	Small/medium	No	**Shareable URL + iframe** from a hosted GEXF (verified via search summary of the Retina pages)	Open source (licence not checked), free	15 min	Attribute filters, color/size mapping
Neo4j Bloom / NVL	Browser	Bloom is UI-limited; NVL is a renderer	Bloom: none native (UNVERIFIED)	NVL React exists, but **NVL's licence allows use only with Aura/Neo4j commercial products** (verified LICENSE)	Bloom included with Aura tiers (UNVERIFIED on Free)	1 h	Perspectives, rule-based styling
Memgraph Lab (Community)	Browser	**Defaults: 3,500 nodes / 8,500 edges** rendered (verified docs; configurable)	No	Desktop/Docker app, not embeddable	Community free (verified)	30 min (Docker)	GSS styling, Cypher good for "ask the graph"
Kepler.gl (geo)	Browser GPU (deck.gl)	Millions of points/arcs (vendor)	**Time playback filter** (vendor docs, not re-checked)	React component	**MIT** (pushed Oct 17, 2026)	1–2 h	**GDELT locations:** arcs between LatAm countries co-mentioned with AI topics over time
deck.gl (ArcLayer/GraphLayer)	Browser GPU	Millions (vendor)	Custom	React	MIT (not re-checked)	3+ h	Fully custom

Tool	GPU where	Interactive scale (quoted)	Temporal animation	Embed in Lovable/Netlify	License / cost	Setup	Explainability affordances
G6 / AntV	Browser (Canvas/WebGL)	Tens of thousands (vendor claims vary)	Timebar plugin (vendor; not re-checked)	npm/React	**MIT** (pushed Oct 16, 2026)	2–4 h	Rich built-in tooltips and plugins
react-force-graph (2D/3D)	Browser (Canvas/WebGL via three.js)	~10k nodes smooth in 3D (folklore)	Custom (update data)	React	**MIT** (pushed Sep 29, 2026)	1 h	Easy click handlers; 3D wow factor; less legible
Flourish / D3	Browser (SVG/Canvas)	Small	**Animated story steps** (Flourish "story" mode; D3 custom)	Flourish embed / D3 in React	Flourish free for public projects (not re-checked); D3 ISC	1–2 h	Best for a narrated "topic birth" sequence of 5–8 frames

2.5 Recommendation: primary and backup per tier

Tier 1, GPU wow (the whole 90-day temporal graph):

- **Primary: Graphistry Hub.**
  - Upload `viz_edges_full` (90 days, n ≥ 3; src, dst, first_dt, wk, n, npmi, mean_tone) and `viz_nodes_full` (etype, n_docs, mean_tone, mean_arousal, fear/hope densities, Leiden community from cuGraph or igraph, evidence URL string) with PyGraphistry.
  - Encode colors: diverging palette for tone/arousal; categorical for community.
  - Embed the share URL in the Lovable app (iframe) and use the timebar on `wk` / `first_dt`.
  - **GPU location:** Graphistry's cloud does layout; the browser renders.
  - **Risks:**
  - The Hub free tier is shared, with "resource limits".
  - Free-tier dataset privacy is UNVERIFIED. GDELT is public, so this is acceptable.
  - Venue Wi-Fi.
  - **Mitigation:** record a 60-second screen capture as a fallback.
- **Backup: cosmos.gl** with precomputed coordinates.
  - On a Modal L4: build weekly cumulative graphs, run `cugraph.force_atlas2(max_iter=100, pos_list=previous_week)`, Leiden and PageRank.
  - Write `positions_wk.arrow` and the JSON to Netlify. cosmos.gl renders with the simulation off and v3 transitions between weeks.
  - **GPU location:** our GPU computes once; the client GPU only renders.
  - **Risk:** demo laptop with an integrated GPU. Test on the actual laptop by 09:00 with Chrome hardware acceleration on. Cap at 300k edges, or 150k for an integrated GPU.

Tier 2, explainable story (top-K, evidence on click):

- **Primary: sigma.js v3 + graphology** in the React app, fed by `viz_nodes_weekly` / `viz_edges_weekly` (S10) exported as static JSON per week.
  - Coordinates come from cuGraph or graphology FA2, warm-started so frames are stable.
  - Interactions:
  - **Week slider** with tweened transitions.
  - **Node fill** = mean tone (red↔green) or relative arousal.
  - **Node ring** = fear vs. hope.
  - **Edge width** = n; **edge color** = edge tone; **tooltip** = n, NPMI, first week seen.
  - **New edges** pulse.
  - **Leiden community** hulls/colors, with **birth/merge** badges from community matching (Jaccard ≥ 0.3).
  - **Click a node** to open a side panel: n_docs, tone, arousal, fear/hope, ES/PT share, and 3 evidence headlines with links and tone.
- **Backup:** react-force-graph-2d with the same JSON (fastest to wire), or the Graphistry point inspector if the frontend slips.
- **Geo add-on (optional, 1–2 h):** Kepler.gl arcs over time for LatAm-country co-mentions with AI subtopics, from V2Locations country codes. High visual impact for a LatAm jury; build it only if ahead at 06:00.

2.6 Demo storyboard: "AI agents", last 90 days (~75 seconds)

1. **Wow (15 s):** the Graphistry full graph of 90 days of AI news in 65 languages. The timebar sweeps July → October and the whole graph lights up. *"Each dot is a company, person or place in AI news; color is the emotion of the coverage."*
2. **Zoom to the story (10 s):** cut to the sigma.js story view, ego-graph of **"agents"**, week 1.
3. **Time slider (20 s):** step week by week.
   - New org/person/theme connections pulse in: e.g., agents × enterprise software vendors × "jobs" × Colombia/México.
   - A **Leiden community is born** and later **merges** with the "enterprise adoption" cluster (badge).
   - Nodes shift from neutral to **high-arousal orange**: fear and hope rising together.
4. **Parallel chart (15 s):**
   - Under the graph, two lines: **share of AI coverage** (normalized volume, rolling 7d) and **Emotional charge** (tone extremity + controversy + GCAM arousal relative to all news).
   - A vertical **"we flagged it here"** marker sits where z_7d crossed the threshold, N days before the volume peak, with the backtest lead time printed.

5. **Click evidence (10 s):** click the node for a Colombian or Mexican outlet. The side panel shows 3 headlines (es/pt), tone, fear/hope densities and the framing angle **"Debate"**.
6. **Adapt (5 s):** the button "Ver brief en español" opens the adaptation brief.

All example entities above are placeholders. The real ones come from the data on the night.

---

3. Graph DB for the demo: minimal path

Option	Limits (verified)	Pros	Cons	Verdict
None: Parquet → igraph/cuGraph → Graphistry/sigma JSON	—	Fewest moving parts; computation already happens in DuckDB/igraph/cuGraph	No Cypher moment	**Default**
Memgraph (Docker on the box) + Lab Community	Lab renders 3,500 nodes / 8,500 edges by default (configurable); in-memory DB	Free; Cypher; quick CSV load; PyGraphistry has a Memgraph connector	Not embeddable; another service to babysit	**Optional, 30 min**, for a "query the graph" moment in Q&A
Neo4j AuraDB Free	**200,000 nodes / 400,000 relationships, 1 GB storage**, GCP us-central1, pauses after 72 h idle (verified, Neo4j docs "quotas and limits")	Hosted; Bloom (UNVERIFIED on Free); familiar brand	Our 90-day full graph (≤ 0.33M nodes) **exceeds** the Free limits, so only the pruned graph fits. NVL licence restricted to Neo4j products	Fallback only

An older Neo4j marketing page quotes 50k nodes / 175k relationships for AuraDB Free. The current docs say 200k / 400k; we use the docs.

Recommendation:

- No graph DB in the critical path.
- If someone has spare time after 06:00, load the pruned 90-day graph (≤ ~20k nodes) into Memgraph on the box and prepare 3 Cypher queries, e.g. "shortest path between OpenAI and Colombia's MinTIC via AI-regulation coverage".
- Kuzu remains archived (Oct 10, 2025).

---

4. Temporal graph model: deeper plan

4.1 Libraries (verified today via GitHub API and arXiv)

Library	Status	Models	Use
TGM (`tgm-team/tgm`, `pip install tgm-lib`), arXiv:2510.07586 "TGM: a Modular and Efficient Library for Machine Learning on Temporal Graphs"	MIT, **beta**, pushed Sep 22, 2026	EdgeBank, GCN, GC-LSTM, GraphMixer, TGAT, TGN, DyGFormer, TPNet, ROLAND, **PopTrack**, TNCN, CTAN (README)	**Primary**: one API for continuous- and discrete-time, includes baselines; claims ~7.8× faster training
TGB (`shenyangHuang/TGB`, `py-tgb` 2.3.0, Jul 27, 2026)	MIT, maintained	Datasets + **evaluators and negative samplers**	Evaluation protocol (MRR with sampled negatives)
DyGLib / DyGLib_TGB (yule-BUAA)	MIT, **last push 2023**	DyGFormer + 8 baselines	Fallback only (unmaintained)
PyG `TGNMemory` example	Maintained with PyG (not re-checked today)	TGN	Fallback minimal TGN
T-GNNExplainer (ICLR 2023, OpenReview BR_ZhvcYbGJ)	Only **community reproductions** (`cisaic/tgnnexplainer`, 2024; `m-krastev/fact8-temporal-graph`, GPL-3.0, 2024)	MCTS explainer over past events	**Not** for the hackathon (setup risk); cite it as future work

4.2 Task framing

(A) Future link prediction, entity–topic:

- Bipartite continuous-time graph: entity (org/person/location/theme) ↔ topic (11 subtopics + Leiden communities).
- An event = an article published at time t that mentions both.
- Edge features: article tone, polarity, GCAM arousal, fear/hope density, language, LatAm flag.
- Question: *"which entities will attach to topic X next week?"*
- **Metric:** MRR using TGB-style one-vs-many evaluation. Each positive is ranked against sampled negatives (random + **historical**) with the TGB negative-sampler API, reported **separately for new edges** (pair never seen before). New edges are the ones that matter for emergence.

(B) Node traction (discrete-time, weekly):

- Predict next-week normalized share growth of each topic/community node; TGB `nodeproppred`-style.
- **Metric:** NDCG@10 and Spearman, compared with LightGBM on interpretable features.

4.3 Data prep

1. **From BigQuery:** `article_topic` + `article_entities` + `gcam_ai` + `gkg_ai.tone` for the window, exported to Parquet (free `tabledata.list` / extract).
2. **Events table:** `(src_id, dst_id, ts, label=1, feat[...])`, sorted by ts.
    - Entities with < 20 documents in the window are dropped, keeping the top ~5–10k (estimate).

- **90 days:** ~0.42M articles → ~2–4M entity–topic events after pruning (estimate). The full co-mention stream (~7M) is used only for EdgeBank and the graph features.
- **2 years:** ~25–40M events (estimate). Hackathon scope = **90 days**, with the 2-year run on the roadmap.

3. **Chronological split 70/15/15**, with the 2025 outage masked when using 2 years.
4. Negatives come from the TGB samplers (historical + random). Seeds are fixed.

4.4 Baselines (must beat to ship)

- **EdgeBank** (∞ and time-window variants; arXiv:2207.10128).
- **Persistence** (last week's links / last week's traction).
- **PopTrack** (popularity-tracking baseline in TGM).
- **LightGBM + SHAP** on interpretable weekly graph features: degree growth, new-neighbour rate, PageRank delta, burst, z_7d, emotional charge.
- **GraphMixer** (arXiv:2302.11636) as the "simple neural" bar.

4.5 Compute (estimates)

Run	GPU	Time (estimate)	Cost
EdgeBank / PopTrack / persistence	CPU	minutes	$0
GraphMixer, 90 days (~3M events), 5 epochs × 3 seeds	Modal L4 ($0.80/h)	0.5–1.5 h	$0.4–1.2
TGN, 90 days, 5 epochs × 3 seeds	Modal A10 ($1.10/h)	1–3 h	$1.1–3.3
DyGFormer, 90 days	Modal A10 / H100 ($3.95/h Modal; $3.19/h Hyperbolic)	2–5 h on A10; ~1–2 h on H100	$2–8
cuGraph FA2 + Leiden for 13 weekly snapshots	Modal L4	5–15 min	< $0.5

Modal and Hyperbolic prices were verified earlier today (Hyperbolic via a third-party tracker). Run times are estimates and depend on TGM's beta performance and on neighbor-sampling settings.

4.6 Explainability for the TGNN

1. **Top contributing past interactions (model-agnostic, cheap):** for a flagged prediction (entity e → topic X), take the K = 20 most recent temporal neighbors in the model's sampler. Remove each one and re-score; rank by score drop. Show the top 3 as evidence articles in the story view.
2. **Attention weights** (TGAT/TGN temporal attention, DyGFormer patches): useful as a heatmap. Attention ≠ explanation, so it is shown only alongside (1).
3. **SHAP on LightGBM** stays the primary "why" for the ranking.
4. T-GNNExplainer: future work (no maintained official code).

4.7 Honest go/no-go (decide at 07:00, final at the 08:00 cut line)

GO for showing the TGNN in the demo only if all of these hold:

1. On the test split, **new-edge MRR** ≥ 1.10 × the best of EdgeBank, PopTrack and persistence, with mean ± sd over 3 seeds, and the improvement > 2 sd.
2. End-to-end reproducible (one command) and finished by 07:00.
3. The top-interaction explanation works for at least the top 3 flagged edges.
4. It doesn't degrade the LightGBM traction ranking when used as an extra feature (NDCG@10 not lower).

Otherwise: show the **baseline table slide**: "We trained TGN/GraphMixer on 3M temporal events; EdgeBank/PopTrack were within x%, so our radar ranks with interpretable features + SHAP." This is still a credibility win (see `gdelt_graph_analysis.md` §8).

---

5. How GCAM emotions and the graph combine

- **Node encoding:**
  - fill = tone, or GCAM arousal relative to the same-day all-news baseline;
  - ring = log(fear_pk / hope_pk), where red = fear-led and green = hope-led;
  - size = n_docs;
  - halo = high-arousal share.
- **Edge encoding:** color = mean tone of co-mentioning articles. **Emotion-shift edge signal** = Δ edge tone (or Δ fear density) week over week. A strongly shifting edge, such as "agents × jobs" turning fearful, is a story cue.
- **Community emotion:** the weighted mean per Leiden community per week. Community birth with high arousal is the strongest "about to land" cue (hypothesis to backtest).
- **Into the six scores: Emotional charge** (score 6) becomes:

`mean(z(extremity_7d), z(polarity_7d), z(controversy_7d), |z(tone_shift_7d)|, z(arousal_rel_7d), z_fear_and_hope_7d)`

```
 - The two GCAM terms come from `gcam_topic_rolling`.
- The equal weights are placeholders until the §10.7 validation in MASTER_PLAN.
```

- **Framing angle:** GCAM sharpens it.

    - fear_rel > 1.3 → "Alerta / riesgo".
    - hope_rel > 1.3 → "Oportunidad".
    - both > 1.2 → **"Debate"**.
    - uncertainty_rel > 1.3 → "Qué significa para tu empresa" (an explainer for executives).
    - The thresholds are placeholders, to be set at P80 of the AI-topic distribution.
- **TGNN edge features:** tone, arousal, fear and hope go in as event features, so the model can learn whether emotionally charged co-mentions precede new links. We test this, not assume it.
- **Caveat:** news emotion ≠ audience emotion. Validation stays as in MASTER_PLAN §10.7.

---

6. Budget: the $250 Google Cloud credit plus other providers

Item	Provider / billing	Estimate	Notes
Graph-ready 2-year backfill (S2), daily_totals, aggregates	BigQuery (free 1 TiB)	$0–3	0.42–0.61 TiB + ~0.1 TiB
GCAM 108 days (demo)	BigQuery (credit)	**~$1.4–1.9**	4 monthly runs
GCAM remaining 20 months	BigQuery (credit)	**~$8–12**	Overnight
GCAM-related own-table reads, S9/S10 rebuilds	BigQuery	~$1–2	
Daily S6 + S7d after the hackathon	BigQuery	~$1.5/month	
Storage of own tables (gkg_ai, gcam_ai, aggregates)	BigQuery storage	~$0.3–0.7/month	~20–40 GB logical (estimate)
Subtotal on the GCP credit (hackathon)		**≈ $12–20**	**≤ 8% of $250**, leaving >$200 for a post-hackathon month or a GCP GPU VM (GPU quota on new projects often 0; UNVERIFIED)
cuGraph layouts + TGNN runs	Modal (separate billing)	$5–15	L4/A10
LLM labeling (6 months, Qwen3-30B)	Nebius Token Factory (pending confirmation)	~$19	
Jev decisions	OpenRouter	~$1–2	
X counts	X API	~$1–3	
Graphistry Hub	Free tier	$0	Resource limits
sigma.js / cosmos.gl / Netlify / Lovable	Free / OSS	$0	
Weekend total, all providers		**≈ $40–60**	Estimate

---

7. Unverified / to confirm

- Whether the $250 credit covers BigQuery on-demand analysis (check credit scope).
- Actual GCAM bytes per month: dry-run first. Older months may be 10–20% larger.
- Graphistry Hub free-tier privacy (public vs. private datasets), exact resource limits, and timebar controls in the embed.
- cosmos.gl v3 parameter names for fixed positions, and whether its React wrapper exists for v3.
- Cosmograph app timeline details.
- sigma.js comfortable scale on the demo laptop: measure it.
- AuraDB Free including Bloom; the AuraDB docs vs. old marketing limits (we use the docs: 200k / 400k).
- TGM beta stability and speed on our data; the PyG TGN example status.
- All TGNN and layout run times (estimates).
- Graph sizes beyond the PoC (Heaps extrapolation; upper bounds).
- Kepler.gl, G6 and Flourish feature claims (vendor, not re-checked today).
- Hyperbolic H100 price (third-party tracker).
- Terms for redistributing GCAM-derived scores.

8. Files

- **This analysis:** `graph_viz_tgnn_gcam.md` / `.html` / `.pdf`.
- **SQL:**
    - `bq_sql/s7_gcam_monthly.sql` (monthly GCAM parse → `gcam_ai` + `gcam_daily_baseline`)
    - `bq_sql/s7d_gcam_daily.sql` (daily incremental)
    - `bq_sql/s9_gcam_topic.sql` (`gcam_daily_topic` + `gcam_topic_rolling` view)
    - `bq_sql/s10_graph_viz_export.sql` (pruned, emotion-annotated weekly graph + evidence)
    - Zipped in `bq_sql.zip`.
- **Measurements:**
    - `gdelt_poc/graph_size.py` + `graph_size_out.txt` (graph sizes and Heaps fit)
    - `gdelt_poc/gcam_coverage_out.txt` (GCAM dimension coverage)